DEVSPHERE

INTERNSHIP PROGRAM

WEEK 6 FINAL PROJECT REPORT

Artificial Intelligence (AI) & Machine Learning (ML)

Intern	**Saud Masood**
Program	DevSphere Internship Program
Domain	Artificial Intelligence & Machine Learning
Projects	AI Student Success System; ML House Price Prediction
Report Date	15 October 2026

Prepared for internship task submission

Report Contents

Report Contents 2
1. Project Overview 3
Requirements Coverage 3
2. AI Week 6 — Intelligent Student Success System 4
2.1 Objective 4
2.2 Dataset fields 4
2.3 Method 4
3. AI Source Code 4
4. AI Output and Screenshot Sections 18
5. ML Week 6 — House Price Prediction System 24
5.1 Objective 24
5.2 Dataset fields 24
5.3 Method 24
6. ML Source Code 24
7. ML Output and Screenshot Sections 36
8. Generated Files and Execution Notes 42
AI project files 42
ML project files 42
Execution notes 42
Conclusion 42
Submission Details 42

1. Project Overview

This report documents the Week 6 AI and ML final projects for the DevSphere Internship Program. The AI project is a student outcome classification and recommendation demonstration. The ML project is a house-price regression demonstration. Both include data preprocessing, model training, evaluation, visualizations, and interactive user input.

Both scripts generate synthetic datasets internally. No real student or property dataset was supplied. Consequently, results must be taken from the actual Google Colab run, and predictions must not be presented as validated real-world outcomes or market valuations.

Requirements Coverage

Project	Implementation	Evidence to Add
AI	Preprocessing, ML models, MLP neural network, tuning, predictions, recommendations and charts	Actual Colab screenshots
ML	Preprocessing, three regression models, evaluation metrics, charts and user-input prediction	Actual Colab screenshots

2. AI Week 6 — Intelligent Student Success System

2.1 Objective

Predict a simulated Pass/Fail outcome using study hours, attendance, and previous exam score. The project compares Logistic Regression, Decision Tree, Random Forest, and an MLP neural network. It tunes Random Forest using GridSearchCV and produces rule-based study recommendations.

2.2 Dataset fields

- StudyHours — simulated daily study hours.
- Attendance — simulated attendance percentage.
- PreviousScore — simulated previous exam score.
- Passed — generated binary label (0 = Fail, 1 = Pass).

The script generates 600 synthetic rows and saves student_performance_week6.csv. A formula with random noise creates the target labels. The data is for demonstrating a workflow, not for drawing conclusions about actual students.

2.3 Method

- Split data into training and testing sets (80/20).
- Impute missing numeric feature values using the training-set median within each pipeline.
- Scale features for Logistic Regression and MLP.
- Train four classifiers and evaluate Accuracy, Precision, Recall, and F1 Score.
- Tune Random Forest with GridSearchCV and three-fold cross-validation.
- Visualize model metrics, confusion matrix, feature importance, and feature relationships.
- Accept student input, display predicted class probabilities, and generate rule-based recommendations.

3. AI Source Code

CODE:

```
# ============================================================
# DEVSPHERE INTERNSHIP - WEEK 6 FINAL PROJECT
# AI INTELLIGENT SYSTEM
# Student Success Prediction and Recommendation System
# ============================================================

import warnings
warnings.filterwarnings("ignore")

import numpy as np
```

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neural_network import MLPClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay
)

# ============================================================
# 1. PROJECT HEADER
# ============================================================

print("=" * 65)
print("      DEVSPHERE INTERNSHIP - WEEK 6 FINAL PROJECT")
print("      AI STUDENT SUCCESS INTELLIGENT SYSTEM")
print("=" * 65)

print("\nProject: Student Performance Prediction and Recommendation")
print("Developer: Saud Masood")
print("Domain: Artificial Intelligence and Machine Learning")

# ============================================================
# 2. CREATE EDUCATIONAL DATASET
# ============================================================
```

```
print("\n[1/8] Creating student dataset...")

np.random.seed(42)

n = 600

study_hours = np.random.uniform(0.5, 10, n)
attendance = np.random.uniform(40, 100, n)
previous_score = np.random.uniform(30, 100, n)

# Synthetic educational score used to generate demonstration labels.
# These are simulated data, not real student records.
performance_score = (
  4.0 * study_hours
  + 0.30 * attendance
  + 0.35 * previous_score
  + np.random.normal(0, 7, n)
)

passed = (performance_score >= 48).astype(int)

df = pd.DataFrame({
  "StudyHours": np.round(study_hours, 2),
  "Attendance": np.round(attendance, 2),
  "PreviousScore": np.round(previous_score, 2),
  "Passed": passed
})

# Add a small number of missing feature values for preprocessing practice.
for column in ["StudyHours", "Attendance", "PreviousScore"]:
  missing_indices = df.sample(
    frac=0.02, random_state=42
  ).index
  df.loc[missing_indices, column] = np.nan

df.to_csv("student_performance_week6.csv", index=False)

print("Dataset created successfully!")
print("Dataset shape:", df.shape)
```

```
print("\nFirst 10 rows:")
print(df.head(10))

print("\nMissing values before preprocessing:")
print(df.isnull().sum())

print("\nClass distribution:")
print(df["Passed"].value_counts().rename(
  index={0: "Fail", 1: "Pass"}
))

# ================================================================
# 3. DATA PREPROCESSING
# ================================================================

print("\n[2/8] Preprocessing dataset...")

X = df[["StudyHours", "Attendance", "PreviousScore"]]
y = df["Passed"]

# Separate training and testing data before fitting preprocessing.
X_train, X_test, y_train, y_test = train_test_split(
  X,
  y,
  test_size=0.20,
  random_state=42,
  stratify=y
)

# Each pipeline imputes missing values using training data only.
# Scaling is used for models that benefit from standardized features.

logistic_pipeline = Pipeline([
  ("imputer", SimpleImputer(strategy="median")),
  ("scaler", StandardScaler()),
  ("model", LogisticRegression(max_iter=2000))
])

tree_pipeline = Pipeline([
  ("imputer", SimpleImputer(strategy="median")),
```

```
    ("model", DecisionTreeClassifier(
        max_depth=5, random_state=42
    ))
])

forest_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("model", RandomForestClassifier(random_state=42))
])

neural_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("model", MLPClassifier(
        hidden_layer_sizes=(32, 16),
        activation="relu",
        solver="adam",
        max_iter=1500,
        early_stopping=True,
        random_state=42
    ))
])

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
print("Missing-value handling: Median imputation")
print("Feature scaling: StandardScaler where appropriate")
print("Preprocessing completed!")

# ============================================================
# 4. TRAIN MULTIPLE ML MODELS AND NEURAL NETWORK
# ============================================================

print("\n[3/8] Training machine learning models...")

models = {
    "Logistic Regression": logistic_pipeline,
    "Decision Tree": tree_pipeline,
    "Random Forest": forest_pipeline,
    "Neural Network (MLP)": neural_pipeline
```

```
}

predictions = {}
results = []

for name, model in models.items():

  model.fit(X_train, y_train)

  y_pred = model.predict(X_test)
  predictions[name] = y_pred

  results.append({
    "Model": name,
    "Accuracy": accuracy_score(y_test, y_pred),
    "Precision": precision_score(
      y_test, y_pred, zero_division=0
    ),
    "Recall": recall_score(
      y_test, y_pred, zero_division=0
    ),
    "F1 Score": f1_score(
      y_test, y_pred, zero_division=0
    )
  })

  print(name, "trained successfully!")

results_df = pd.DataFrame(results)

print("\nInitial model evaluation:")
print(results_df.round(3).to_string(index=False))

# ================================================================
# 5. HYPERPARAMETER TUNING
# ================================================================

print("\n[4/8] Tuning Random Forest with GridSearchCV...")

tuning_pipeline = Pipeline([
```

```
    ("imputer", SimpleImputer(strategy="median")),
    ("model", RandomForestClassifier(random_state=42))
])

param_grid = {
    "model__n_estimators": [50, 100],
    "model__max_depth": [None, 5, 10],
    "model__min_samples_split": [2, 5]
}

grid_search = GridSearchCV(
    estimator=tuning_pipeline,
    param_grid=param_grid,
    scoring="accuracy",
    cv=3,
    n_jobs=-1
)

grid_search.fit(X_train, y_train)

optimized_model = grid_search.best_estimator_

print("\nBest Parameters:")
print(grid_search.best_params_)

print(
    f"\nBest Cross-Validation Accuracy: "
    f"{grid_search.best_score_ * 100:.2f}%"
)

optimized_predictions = optimized_model.predict(X_test)

optimized_accuracy = accuracy_score(
    y_test, optimized_predictions
)
optimized_precision = precision_score(
    y_test, optimized_predictions, zero_division=0
)
optimized_recall = recall_score(
    y_test, optimized_predictions, zero_division=0
```

```
)
optimized_f1 = f1_score(
  y_test, optimized_predictions, zero_division=0
)

print("\nOptimized Random Forest Test Results:")
print(f"Accuracy : {optimized_accuracy * 100:.2f}%")
print(f"Precision: {optimized_precision * 100:.2f}%")
print(f"Recall   : {optimized_recall * 100:.2f}%")
print(f"F1 Score : {optimized_f1 * 100:.2f}%")

print("\nClassification Report:")
print(classification_report(
  y_test,
  optimized_predictions,
  target_names=["Fail", "Pass"],
  zero_division=0
))

# ================================================================
# 6. VISUALIZE MODEL PERFORMANCE
# ================================================================

print("\n[5/8] Generating visualizations...")

# Model comparison chart
metrics = ["Accuracy", "Precision", "Recall", "F1 Score"]

results_df.set_index("Model")[metrics].plot(
  kind="bar",
  figsize=(12, 6),
  ylim=(0, 1.05),
  width=0.8
)

plt.title("Week 6 - Machine Learning Model Comparison")
plt.ylabel("Score")
plt.xlabel("Model")
plt.xticks(rotation=20, ha="right")
plt.legend(loc="lower right")
```

```
plt.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.show()

# Optimized model confusion matrix
cm = confusion_matrix(
  y_test,
  optimized_predictions,
  labels=[0, 1]
)

disp = ConfusionMatrixDisplay(
  confusion_matrix=cm,
  display_labels=["Fail", "Pass"]
)

disp.plot(cmap="Blues")
plt.title("Optimized Random Forest - Confusion Matrix")
plt.tight_layout()
plt.show()

# Feature importance from optimized Random Forest
forest_model = optimized_model.named_steps["model"]

importance = pd.Series(
  forest_model.feature_importances_,
  index=X.columns
).sort_values(ascending=False)

plt.figure(figsize=(8, 5))
importance.plot(kind="bar")
plt.title("Feature Importance - Optimized Random Forest")
plt.xlabel("Student Feature")
plt.ylabel("Importance")
plt.xticks(rotation=20, ha="right")
plt.tight_layout()
plt.show()

# Visualize generated dataset
plt.figure(figsize=(8, 5))
```

```
sns.scatterplot(
  data=df,
  x="StudyHours",
  y="PreviousScore",
  hue="Passed",
  palette={0: "red", 1: "green"}
)
plt.title("Study Hours vs Previous Score")
plt.xlabel("Study Hours")
plt.ylabel("Previous Exam Score")
plt.legend(title="Outcome", labels=["Fail", "Pass"])
plt.tight_layout()
plt.show()

# ============================================================
# 7. STUDENT INPUT AND PREDICTION
# ============================================================

print("\n[6/8] STUDENT PREDICTION SYSTEM")
print("=" * 65)

print("Enter details for a student.")
print("Study hours should be between 0 and 24 per day.")
print("Attendance and previous score should be between 0 and 100.")

try:
  hours = float(input("\nDaily study hours: "))
  attendance_value = float(input("Attendance percentage: "))
  previous_value = float(input("Previous exam score: "))

  if not (0 <= hours <= 24):
    raise ValueError("Study hours must be between 0 and 24.")

  if not (0 <= attendance_value <= 100):
    raise ValueError("Attendance must be between 0 and 100.")

  if not (0 <= previous_value <= 100):
    raise ValueError("Previous score must be between 0 and 100.")

  student = pd.DataFrame({
```

```
    "StudyHours": [hours],
    "Attendance": [attendance_value],
    "PreviousScore": [previous_value]
})

# Use optimized model for the final prediction.
prediction = int(optimized_model.predict(student)[0])
probabilities = optimized_model.predict_proba(student)[0]

pass_probability = probabilities[
    list(optimized_model.classes_).index(1)
] * 100

fail_probability = probabilities[
    list(optimized_model.classes_).index(0)
] * 100

outcome = "PASS" if prediction == 1 else "FAIL"

print("\n" + "=" * 65)
print("          STUDENT PREDICTION RESULT")
print("=" * 65)

print(f"Study Hours: {hours:.1f} per day")
print(f"Attendance: {attendance_value:.1f}%")
print(f"Previous Score: {previous_value:.1f}")
print(f"\nPredicted Outcome: {outcome}")
print(f"Model-estimated Pass Probability: {pass_probability:.2f}%")
print(f"Model-estimated Fail Probability: {fail_probability:.2f}%")

# ------------------------------------------------------------
# 8. PERSONALIZED RECOMMENDATIONS
# ------------------------------------------------------------

print("\n[7/8] PERSONALIZED RECOMMENDATIONS")
print("-" * 65)

recommendations = []

if hours < 2:
```

```
    recommendations.append(
        "Create a regular study schedule and gradually increase study time."
    )
elif hours < 4:
    recommendations.append(
        "Try consistent daily revision and focused study sessions."
    )
else:
    recommendations.append(
        "Maintain your study routine and review difficult topics regularly."
    )

if attendance_value < 60:
    recommendations.append(
        "Improve class attendance and catch up on missed lessons."
    )
elif attendance_value < 80:
    recommendations.append(
        "Aim for more consistent attendance to strengthen understanding."
    )
else:
    recommendations.append(
        "Maintain your good attendance habits."
    )

if previous_value < 50:
    recommendations.append(
        "Review foundational topics and practice questions with feedback."
    )
elif previous_value < 70:
    recommendations.append(
        "Practice past papers and identify topics needing improvement."
    )
else:
    recommendations.append(
        "Continue revision and use practice tests to maintain progress."
    )

if prediction == 0:
    recommendations.append(
```

```
        "Consider asking a teacher for a targeted study plan."
    )
else:
    recommendations.append(
        "Keep monitoring progress; this prediction is not a guarantee."
    )

for i, recommendation in enumerate(recommendations, start=1):
    print(f"{i}. {recommendation}")

# Prediction probability chart
plt.figure(figsize=(7, 5))

bars = plt.bar(
    ["Pass Probability", "Fail Probability"],
    [pass_probability, fail_probability],
    color=["green", "red"]
)

plt.ylim(0, 110)
plt.ylabel("Estimated Probability (%)")
plt.title("Student Outcome Prediction")

for bar, value in zip(
    bars, [pass_probability, fail_probability]
):
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        value + 2,
        f"{value:.1f}%",
        ha="center"
    )

plt.tight_layout()
plt.show()

# Save prediction to a CSV file
prediction_record = student.copy()
prediction_record["PredictedOutcome"] = outcome
prediction_record["PassProbabilityPercent"] = round(
```

```
        pass_probability, 2
    )
    prediction_record["FailProbabilityPercent"] = round(
        fail_probability, 2
    )

    prediction_record.to_csv(
        "student_prediction_result.csv",
        index=False
    )

    print("\nPrediction saved to student_prediction_result.csv")

except ValueError as error:
    print("\nInvalid input:", error)
    print("Run this cell again and enter valid numeric values.")

# ============================================================
# PROJECT COMPLETION
# ============================================================

print("\n[8/8] PROJECT SUMMARY")
print("=" * 65)
print("Dataset: student_performance_week6.csv")
print("Models: Logistic Regression, Decision Tree, Random Forest")
print("Neural Network: Multi-Layer Perceptron (MLP)")
print("Tuning: GridSearchCV")
print("Metrics: Accuracy, Precision, Recall, F1 Score")
print("Visualization: Comparison, Confusion Matrix, Feature Importance")
print("Interaction: Student input, prediction, recommendations")
print("\nWEEK 6 AI INTELLIGENT SYSTEM EXECUTION FINISHED!")
print("=" * 65)
```

4. AI Output and Screenshot Sections

OUTPUT SCREENSHOTS:

```
================================================================
       DEVSPHERE INTERNSHIP - WEEK 6 FINAL PROJECT
       AI STUDENT SUCCESS INTELLIGENT SYSTEM
================================================================

Project: Student Performance Prediction and Recommendation
Developer: Saud Masood
Domain: Artificial Intelligence and Machine Learning

[1/8] Creating student dataset...
Dataset created successfully!
Dataset shape: (600, 4)

First 10 rows:
   StudyHours  Attendance  PreviousScore  Passed
0        4.06       50.14          83.08       1
1        9.53       56.72          31.72       1
2        7.45       50.62          31.55       1
3        6.19       45.32          52.65       1
4        1.98       47.24          64.21       0
5        1.98       67.65          83.93       1
6        1.05       52.38          77.83       1
7        8.73       61.86          61.21       1
8        6.21       70.21          49.15       1
9        7.23       81.42          99.80       1

Missing values before preprocessing:
StudyHours       12
Attendance       12
PreviousScore    12
Passed            0
dtype: int64

Class distribution:
Passed
Pass    508
Fail     92
Name: count, dtype: int64

[2/8] Preprocessing dataset...
Training samples: 480
Testing samples: 120
Missing-value handling: Median imputation
Feature scaling: StandardScaler where appropriate
Preprocessing completed!

[3/8] Training machine learning models...
Logistic Regression trained successfully!
Decision Tree trained successfully!
Random Forest trained successfully!
Neural Network (MLP) trained successfully!

Initial model evaluation:
               Model  Accuracy  Precision  Recall  F1 Score
 Logistic Regression     0.917      0.926   0.980     0.952
       Decision Tree     0.883      0.907   0.961     0.933
       Random Forest     0.908      0.933   0.961     0.947
Neural Network (MLP)     0.875      0.878   0.990     0.931

[4/8] Tuning Random Forest with GridSearchCV...
```

```
Best Parameters:
{'model__max_depth': None, 'model__min_samples_split': 2, 'model__n_estimators': 50}

Best Cross-Validation Accuracy: 91.67%

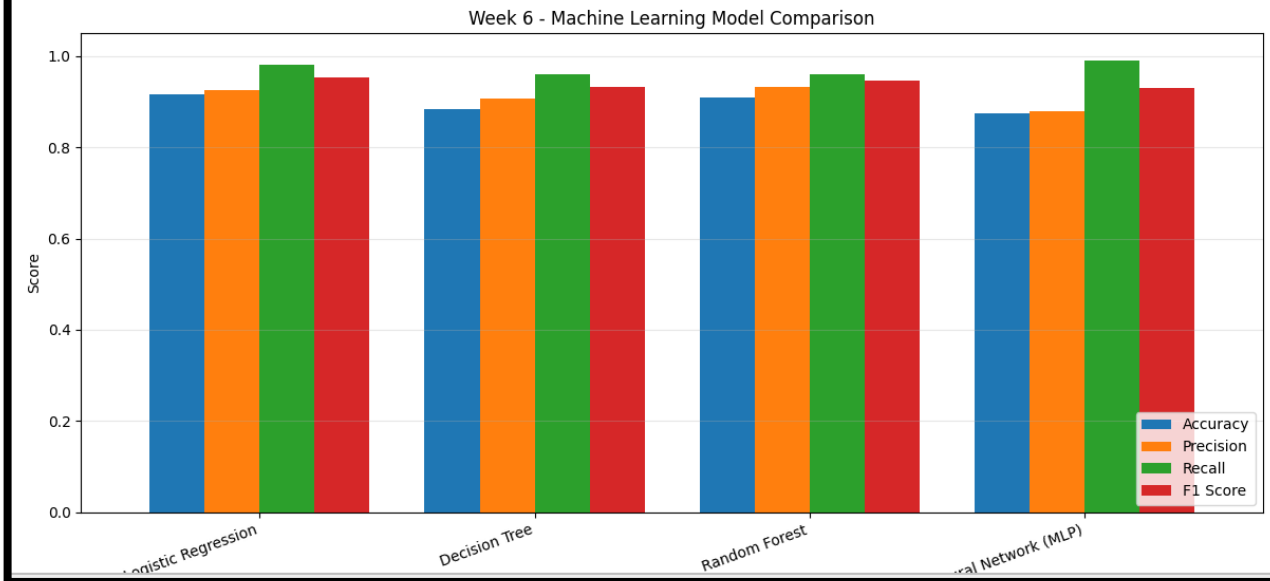
Optimized Random Forest Test Results:
Accuracy : 90.00%
Precision: 92.45%
Recall   : 96.08%
F1 Score : 94.23%

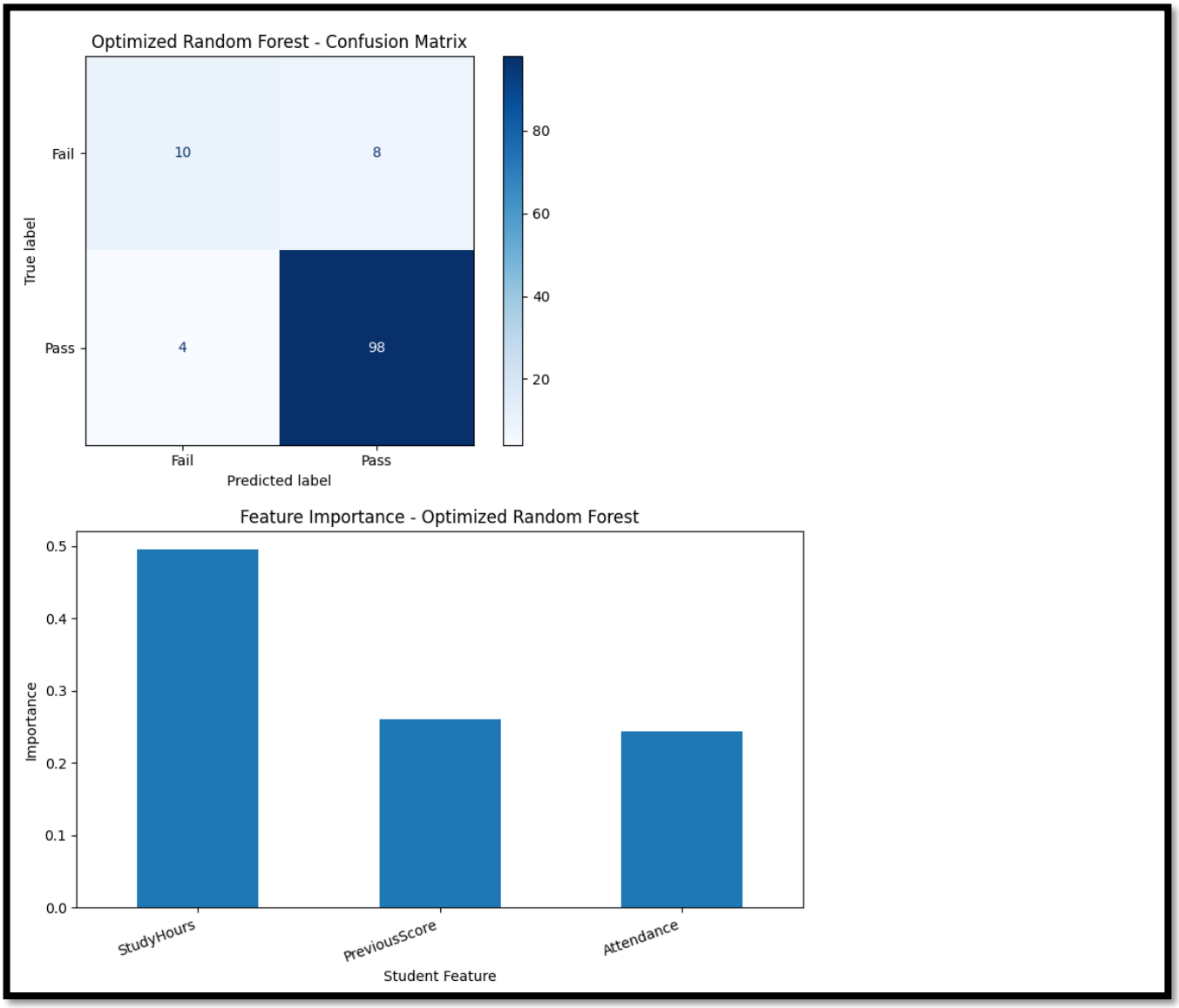
Classification Report:
              precision    recall  f1-score   support

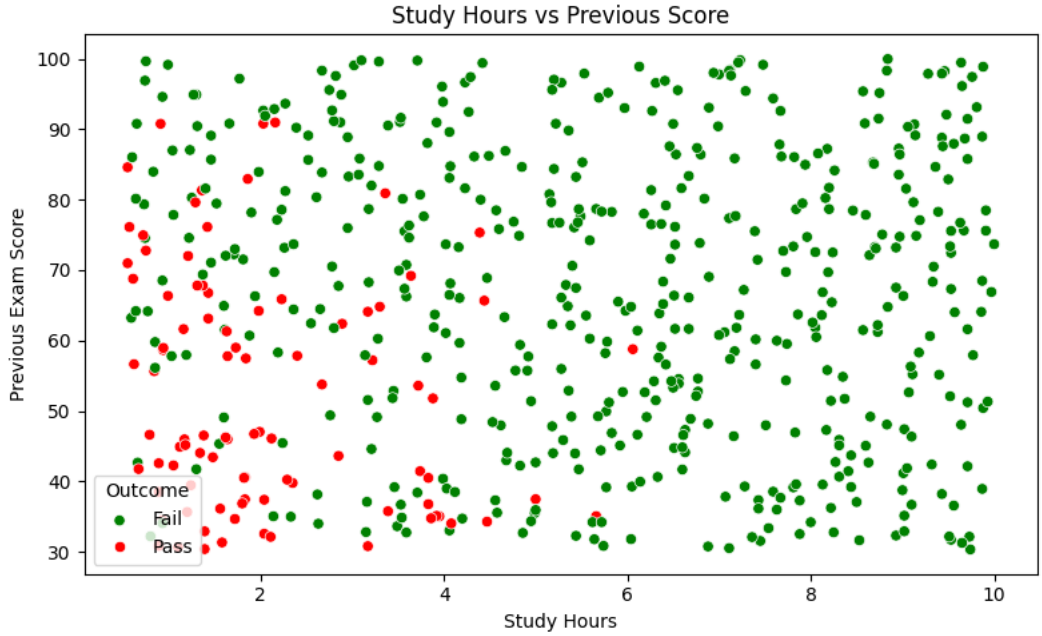
        Fail       0.71      0.56      0.62        18
        Pass       0.92      0.96      0.94       102

    accuracy                           0.90       120
   macro avg       0.82      0.76      0.78       120
weighted avg       0.89      0.90      0.89       120


[5/8] Generating visualizations...
```




Optimized Random Forest - Confusion Matrix
Fail
Pass
True label
10
8
4
98
Fail
Pass
Predicted label
80
60
40
20
Feature Importance - Optimized Random Forest
Importance
0.5
0.4
0.3
0.2
0.1
0.0
StudyHours
PreviousScore
Attendance
Student Feature




```
[6/8] STUDENT PREDICTION SYSTEM
================================================================
Enter details for a student.
Study hours should be between 0 and 24 per day.
Attendance and previous score should be between 0 and 100.

Daily study hours: 4
Attendance percentage: 46
Previous exam score: 23

================================================================
             STUDENT PREDICTION RESULT
================================================================
Study Hours: 4.0 per day
Attendance: 46.0%
Previous Score: 23.0

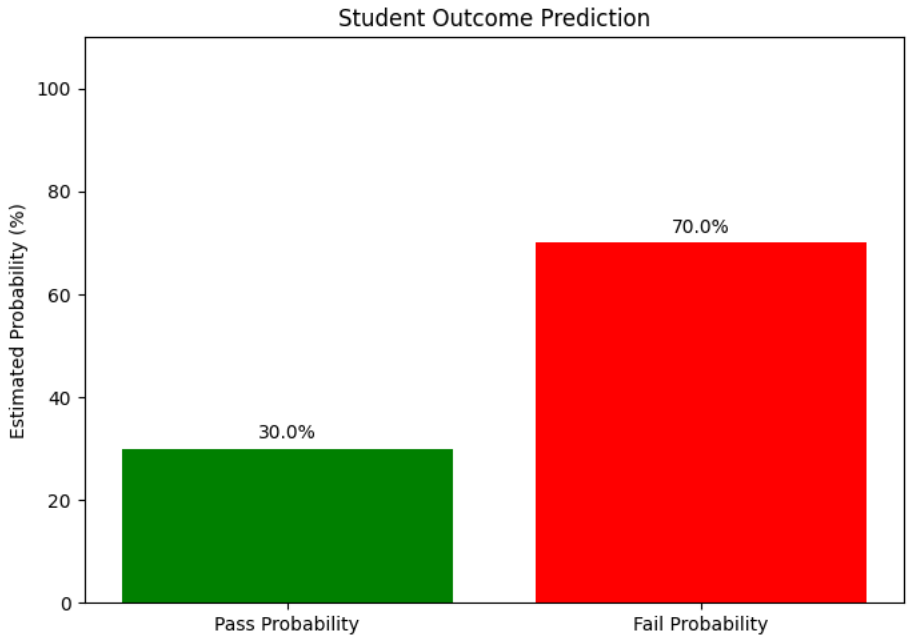
Predicted Outcome: FAIL
Model-estimated Pass Probability: 30.00%
Model-estimated Fail Probability: 70.00%

[7/8] PERSONALIZED RECOMMENDATIONS
----------------------------------------------------------------
1. Maintain your study routine and review difficult topics regularly.
2. Improve class attendance and catch up on missed lessons.
3. Review foundational topics and practice questions with feedback.
4. Consider asking a teacher for a targeted study plan.
```

```
============================================================
             STUDENT PREDICTION RESULT
============================================================
Study Hours: 4.0 per day
Attendance: 46.0%
Previous Score: 23.0

Predicted Outcome: FAIL
Model-estimated Pass Probability: 30.00%
Model-estimated Fail Probability: 70.00%

[7/8] PERSONALIZED RECOMMENDATIONS
------------------------------------------------------------
1. Maintain your study routine and review difficult topics regularly.
2. Improve class attendance and catch up on missed lessons.
3. Review foundational topics and practice questions with feedback.
4. Consider asking a teacher for a targeted study plan.
```




```
Prediction saved to student_prediction_result.csv

[8/8] PROJECT SUMMARY
============================================================
Dataset: student_performance_week6.csv
Models: Logistic Regression, Decision Tree, Random Forest
Neural Network: Multi-Layer Perceptron (MLP)
Tuning: GridSearchCV
Metrics: Accuracy, Precision, Recall, F1 Score
Visualization: Comparison, Confusion Matrix, Feature Importance
Interaction: Student input, prediction, recommendations

WEEK 6 AI INTELLIGENT SYSTEM EXECUTION FINISHED!
============================================================
```

5. ML Week 6 — House Price Prediction System

5.1 Objective

Estimate a house price from area, bedrooms, bathrooms, house age, and distance from the city centre. The project trains Linear Regression, Decision Tree Regression, and Random Forest Regression, compares them using regression metrics, and selects the model with the lowest test RMSE.

5.2 Dataset fields

- Area_sqft — house area in square feet.
- Bedrooms — number of bedrooms.
- Bathrooms — number of bathrooms.
- HouseAge — house age in years.
- Distance_km — distance from city centre in kilometres.
- Price — synthetic target price generated for demonstration.

The script generates 800 synthetic records and saves house_prices_week6.csv. Its price formula is illustrative; it is not real estate market data or a validated valuation model.

5.3 Method

- Remove duplicate records and separate features from the target.
- Split the data into training and testing sets (80/20).
- Impute missing feature values using median imputation in model pipelines.
- Train Linear Regression, Decision Tree Regression, and Random Forest Regression.
- Evaluate with MAE, MSE, RMSE, and R^2 Score.
- Select the model with the lowest test RMSE.
- Plot model comparison, actual versus predicted values, feature analysis, and correlation heatmap.
- Accept user-provided house details and save the estimate to a CSV file.

6. ML Source Code

```
# ============================================================
# DEVSPHERE INTERNSHIP - WEEK 6 FINAL PROJECT
# MACHINE LEARNING: HOUSE PRICE PREDICTION SYSTEM
# ============================================================

import warnings
warnings.filterwarnings("ignore")
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer

from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor

from sklearn.metrics import (
  mean_absolute_error,
  mean_squared_error,
  r2_score
)

# ============================================================
# 1. PROJECT HEADER
# ============================================================

print("=" * 70)
print("      DEVSPHERE INTERNSHIP - WEEK 6 FINAL PROJECT")
print("           HOUSE PRICE PREDICTION SYSTEM")
print("=" * 70)

print("Developer: Saud Masood")
print("Domain: Machine Learning")
print("Task: Complete ML Prediction Project")

# ============================================================
# 2. CREATE DATASET
# ============================================================

print("\n[1/7] Creating house price dataset...")

np.random.seed(42)
```

```
number_of_houses = 800

area = np.random.randint(500, 4001, number_of_houses)
bedrooms = np.random.randint(1, 7, number_of_houses)
bathrooms = np.random.randint(1, 5, number_of_houses)
house_age = np.random.randint(0, 41, number_of_houses)
distance = np.random.uniform(1, 30, number_of_houses)

# Generate synthetic prices for educational demonstration.
# Prices are illustrative, not actual market valuations.

noise = np.random.normal(0, 12000, number_of_houses)

price = (
  25000
  + area * 110
  + bedrooms * 9000
  + bathrooms * 12000
  - house_age * 850
  - distance * 1200
  + noise
)

price = np.maximum(price, 30000)

df = pd.DataFrame({
  "Area_sqft": area,
  "Bedrooms": bedrooms,
  "Bathrooms": bathrooms,
  "HouseAge": house_age,
  "Distance_km": np.round(distance, 2),
  "Price": np.round(price, 2)
})

# Introduce a few missing feature values to demonstrate preprocessing.
for column in [
  "Area_sqft",
  "Bedrooms",
  "Bathrooms",
  "HouseAge",
```

```
    "Distance_km"
]:
    missing_indices = df.sample(
        frac=0.02, random_state=42
    ).index

    df.loc[missing_indices, column] = np.nan

df.to_csv("house_prices_week6.csv", index=False)

print("\nDataset created successfully!")
print("Dataset shape:", df.shape)

print("\nFirst 10 rows:")
print(df.head(10))

print("\nDataset information:")
df.info()

print("\nStatistical summary:")
print(df.describe().round(2))

print("\nMissing values:")
print(df.isnull().sum())

# ============================================================
# 3. DATA PREPROCESSING
# ============================================================

print("\n[2/7] Preprocessing dataset...")

# Remove duplicate rows.
df = df.drop_duplicates().reset_index(drop=True)

# Separate input features and target.
X = df.drop(columns=["Price"])
y = df["Price"]

# Split before fitting the preprocessing pipeline to prevent
# information from the test set leaking into training.
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
print("Duplicates removed.")
print("Missing numerical features will be filled using medians.")
print("Data preprocessing completed!")

# ============================================================
# 4. TRAIN MULTIPLE REGRESSION MODELS
# ============================================================

print("\n[3/7] Training machine learning models...")

models = {
    "Linear Regression": Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("model", LinearRegression())
    ]),

    "Decision Tree": Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("model", DecisionTreeRegressor(
            max_depth=12,
            random_state=42
        ))
    ]),

    "Random Forest": Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("model", RandomForestRegressor(
            n_estimators=150,
            random_state=42,
            n_jobs=-1
        ))
```

```
    ])
}

results = []
trained_models = {}

for name, model in models.items():

    model.fit(X_train, y_train)

    predictions = model.predict(X_test)

    mae = mean_absolute_error(y_test, predictions)
    mse = mean_squared_error(y_test, predictions)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, predictions)

    trained_models[name] = model

    results.append({
        "Model": name,
        "MAE": mae,
        "MSE": mse,
        "RMSE": rmse,
        "R2 Score": r2
    })

    print(f"\n{name} trained successfully!")
    print(f"MAE:  {mae:,.2f}")
    print(f"RMSE: {rmse:,.2f}")
    print(f"R2 Score: {r2:.4f}")

# ============================================================
# 5. MODEL EVALUATION AND COMPARISON
# ============================================================

print("\n[4/7] Comparing model performance...")

results_df = pd.DataFrame(results)
```

```
print("\nModel Comparison:")
print(results_df.round(4).to_string(index=False))

# Select the model with the lowest test RMSE.
best_model_name = results_df.loc[
  results_df["RMSE"].idxmin(),
  "Model"
]

best_model = trained_models[best_model_name]

best_predictions = best_model.predict(X_test)

print("\nBest model based on test RMSE:", best_model_name)

print("\nBest Model Evaluation:")
print(f"MAE: {mean_absolute_error(y_test, best_predictions):,.2f}")
print(f"RMSE: {np.sqrt(mean_squared_error(y_test, best_predictions)):,.2f}")
print(f"R2 Score: {r2_score(y_test, best_predictions):.4f}")

# ============================================================
# 6. VISUALIZATION
# ============================================================

print("\n[5/7] Generating graphs and charts...")

# Chart 1: Compare model RMSE.
plt.figure(figsize=(9, 5))

plt.bar(
  results_df["Model"],
  results_df["RMSE"]
)

plt.title("Model Comparison - RMSE")
plt.xlabel("Regression Model")
plt.ylabel("RMSE (lower is better)")
plt.xticks(rotation=15)
plt.tight_layout()
plt.show()
```

```
# Chart 2: Actual versus predicted house prices.
plt.figure(figsize=(8, 6))

plt.scatter(
  y_test,
  best_predictions,
  alpha=0.65
)

minimum = min(y_test.min(), best_predictions.min())
maximum = max(y_test.max(), best_predictions.max())

plt.plot(
  [minimum, maximum],
  [minimum, maximum],
  linestyle="--"
)

plt.title(f"Actual vs Predicted Prices - {best_model_name}")
plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.tight_layout()
plt.show()

# Chart 3: Feature importance, if supported by the chosen model.
chosen_estimator = best_model.named_steps["model"]

if hasattr(chosen_estimator, "feature_importances_"):

  importance = pd.Series(
    chosen_estimator.feature_importances_,
    index=X.columns
  ).sort_values(ascending=True)

  plt.figure(figsize=(8, 5))
  importance.plot(kind="barh")

  plt.title(f"Feature Importance - {best_model_name}")
  plt.xlabel("Importance")
```

```
    plt.ylabel("House Feature")
    plt.tight_layout()
    plt.show()

else:
    # Use permutation-free coefficient magnitudes for Linear Regression.
    # These are coefficient magnitudes, not causal importance.
    coefficients = pd.Series(
        np.abs(chosen_estimator.coef_),
        index=X.columns
    ).sort_values(ascending=True)

    plt.figure(figsize=(8, 5))
    coefficients.plot(kind="barh")

    plt.title("Absolute Regression Coefficient Magnitudes")
    plt.xlabel("Absolute Coefficient")
    plt.ylabel("House Feature")
    plt.tight_layout()
    plt.show()

# Chart 4: Correlation heatmap.
plt.figure(figsize=(8, 6))

sns.heatmap(
    df.corr(numeric_only=True),
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("House Dataset Correlation Heatmap")
plt.tight_layout()
plt.show()

# ============================================================
# 7. USER INPUT PREDICTION SYSTEM
# ============================================================

print("\n[6/7] HOUSE PRICE PREDICTION")
```

```
print("=" * 70)

print("Enter house details to estimate its price.")
print("Area: 100 to 10000 square feet")
print("Bedrooms: 1 to 20")
print("Bathrooms: 1 to 15")
print("House age: 0 to 150 years")
print("Distance: 0 to 200 km")

try:

  user_area = float(input("\nHouse area (sq ft): "))
  user_bedrooms = int(input("Number of bedrooms: "))
  user_bathrooms = int(input("Number of bathrooms: "))
  user_age = float(input("House age (years): "))
  user_distance = float(input("Distance from city centre (km): "))

  if not 100 <= user_area <= 10000:
    raise ValueError("Area must be between 100 and 10000.")

  if not 1 <= user_bedrooms <= 20:
    raise ValueError("Bedrooms must be between 1 and 20.")

  if not 1 <= user_bathrooms <= 15:
    raise ValueError("Bathrooms must be between 1 and 15.")

  if not 0 <= user_age <= 150:
    raise ValueError("House age must be between 0 and 150.")

  if not 0 <= user_distance <= 200:
    raise ValueError("Distance must be between 0 and 200 km.")

  new_house = pd.DataFrame({
    "Area_sqft": [user_area],
    "Bedrooms": [user_bedrooms],
    "Bathrooms": [user_bathrooms],
    "HouseAge": [user_age],
    "Distance_km": [user_distance]
  })
```

```
predicted_price = best_model.predict(new_house)[0]

print("\n" + "=" * 70)
print("          HOUSE PRICE PREDICTION RESULT")
print("=" * 70)

print(f"Area: {user_area:,.0f} sq ft")
print(f"Bedrooms: {user_bedrooms}")
print(f"Bathrooms: {user_bathrooms}")
print(f"House age: {user_age:.0f} years")
print(f"Distance: {user_distance:.2f} km")

print("\nSelected Model:", best_model_name)
print(f"Estimated House Price: {predicted_price:,.2f}")

print("\nNote: This is an estimate from synthetic training data,")
print("not an actual market valuation.")

# Save prediction.
prediction_record = new_house.copy()
prediction_record["PredictedPrice"] = round(
  predicted_price, 2
)
prediction_record["Model"] = best_model_name

prediction_record.to_csv(
  "house_price_prediction_result.csv",
  index=False
)

print("\nPrediction saved to house_price_prediction_result.csv")

# Display price prediction chart.
plt.figure(figsize=(7, 4))

plt.bar(
  ["Estimated Price"],
  [predicted_price]
)
```

```
    plt.title("Predicted House Price")
    plt.ylabel("Price (dataset units)")
    plt.tight_layout()
    plt.show()

except ValueError as error:

    print("\nInvalid input:", error)
    print("Run this cell again with valid values.")

# ============================================================
# 8. SAVE MODEL COMPARISON AND FINAL SUMMARY
# ============================================================

results_df.to_csv(
    "model_evaluation_results.csv",
    index=False
)

print("\n[7/7] FINAL PROJECT SUMMARY")
print("=" * 70)

print("Dataset: house_prices_week6.csv")
print("Models: Linear Regression, Decision Tree, Random Forest")
print("Preprocessing: Duplicate removal and median imputation")
print("Evaluation: MAE, MSE, RMSE, R2 Score")
print("Visualization: Model comparison, prediction scatter plot,")
print("               feature analysis and correlation heatmap")
print("User interaction: House feature input and price prediction")
print("Saved file: model_evaluation_results.csv")

print("\nML WEEK 6 PROJECT COMPLETED SUCCESSFULLY!")
print("=" * 70)
```

7. ML Output and Screenshot Sections

```
======================================================================
        DEVSPHERE INTERNSHIP - WEEK 6 FINAL PROJECT
              HOUSE PRICE PREDICTION SYSTEM
======================================================================
Developer: Saud Masood
Domain: Machine Learning
Task: Complete ML Prediction Project

[1/7] Creating house price dataset...

Dataset created successfully!
Dataset shape: (800, 6)

First 10 rows:
   Area_sqft  Bedrooms  Bathrooms  HouseAge  Distance_km      Price
0     3674.0       1.0        3.0      30.0         8.99  454183.21
1     1360.0       4.0        2.0      10.0        23.83  185568.00
2     1794.0       1.0        1.0       9.0        29.27  206877.48
3     1630.0       1.0        4.0       2.0        11.77  244021.55
4     1595.0       1.0        4.0      13.0        13.44  229167.93
5     3592.0       6.0        3.0      15.0        25.50  471593.24
6     2138.0       5.0        2.0       9.0         8.78  315890.64
7     2669.0       6.0        2.0      28.0         5.54  372493.13
8      966.0       2.0        2.0       9.0        25.74  122032.83
9     1738.0       4.0        4.0      11.0         2.35  279014.04

Dataset information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 800 entries, 0 to 799
Data columns (total 6 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   Area_sqft    784 non-null    float64
 1   Bedrooms     784 non-null    float64
 2   Bathrooms    784 non-null    float64
 3   HouseAge     784 non-null    float64
 4   Distance_km  784 non-null    float64
 5   Price        800 non-null    float64
dtypes: float64(6)
memory usage: 37.6 KB

Statistical summary:
       Area_sqft  Bedrooms  Bathrooms  HouseAge  Distance_km      Price
count     784.00    784.00     784.00    784.00       784.00     800.00
mean     2315.55      3.49       2.48     19.79        15.70  306154.70
std      1008.48      1.75       1.12     11.71         8.48  115705.63
min       501.00      1.00       1.00      0.00         1.01   37656.14
25%      1476.50      2.00       1.00     10.00         8.14  206813.37
50%      2268.00      3.00       2.00     20.00        16.29  300550.70
75%      3237.25      5.00       3.00     30.00        23.00  408992.98
max      3999.00      6.00       4.00     40.00        29.99  536927.40

Missing values:
Area_sqft      16
Bedrooms       16
Bathrooms      16
HouseAge       16
Distance_km    16
Price           0
dtype: int64
```

```
[2/7] Preprocessing dataset...
Training samples: 640
Testing samples: 160
Duplicates removed.
Missing numerical features will be filled using medians.
Data preprocessing completed!

[3/7] Training machine learning models...

Linear Regression trained successfully!
MAE:  19,535.07
RMSE: 39,366.34
R2 Score: 0.8877

Decision Tree trained successfully!
MAE:  29,091.74
RMSE: 45,822.28
R2 Score: 0.8479

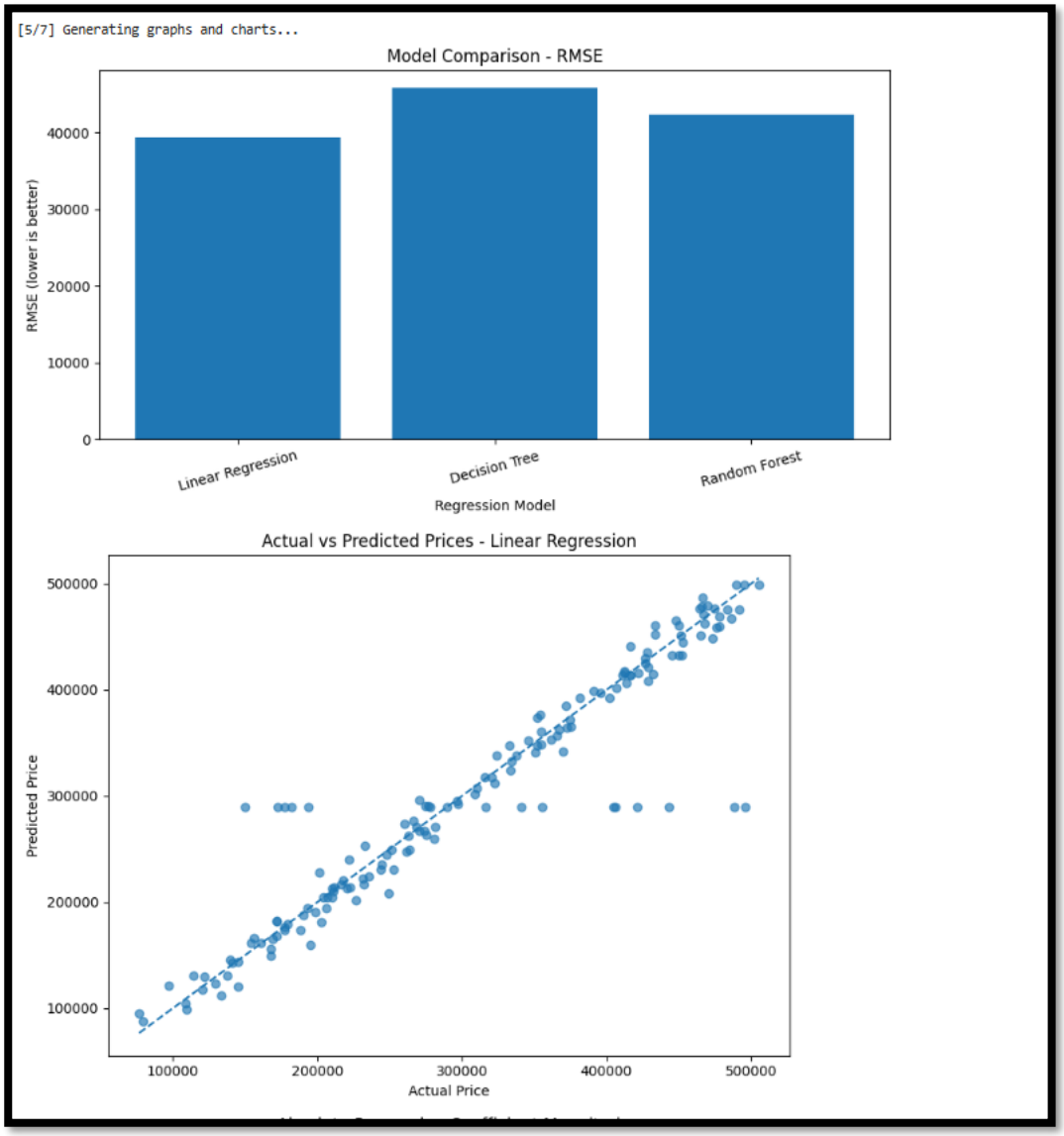
Random Forest trained successfully!
MAE:  23,915.48
RMSE: 42,361.78
R2 Score: 0.8700

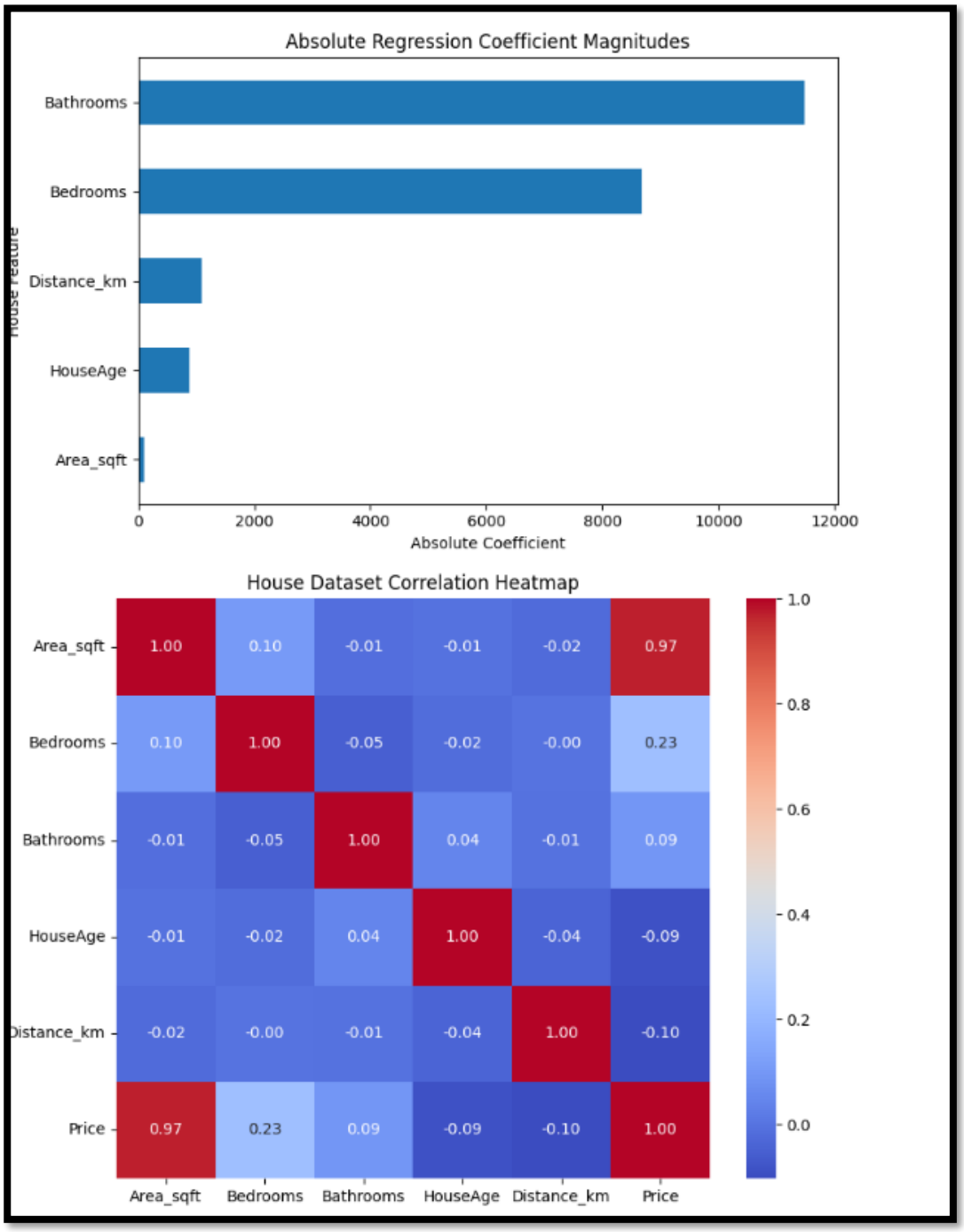
[4/7] Comparing model performance...

Model Comparison:
            Model        MAE          MSE       RMSE  R2 Score
Linear Regression 19535.0713 1.549709e+09 39366.3372    0.8877
    Decision Tree 29091.7383 2.099681e+09 45822.2779    0.8479
    Random Forest 23915.4772 1.794520e+09 42361.7751    0.8700

Best model based on test RMSE: Linear Regression

Best Model Evaluation:
MAE: 19,535.07
RMSE: 39,366.34
R2 Score: 0.8877
```


[5/7] Generating graphs and charts...
Model Comparison - RMSE
RMSE (lower is better)
0
10000
20000
30000
40000
Linear Regression
Decision Tree
Random Forest
Regression Model
Actual vs Predicted Prices - Linear Regression
Predicted Price
100000
200000
300000
400000
500000
Actual Price


Absolute Regression Coefficient Magnitudes
Bathrooms
Bedrooms
Distance_km
HouseAge
Area_sqft
House Feature
0
2000
4000
6000
8000
10000
12000
Absolute Coefficient
House Dataset Correlation Heatmap
Area_sqft 1.00 0.10 -0.01 -0.01 -0.02 0.97
Bedrooms 0.10 1.00 -0.05 -0.02 -0.00 0.23
Bathrooms -0.01 -0.05 1.00 0.04 -0.01 0.09
HouseAge -0.01 -0.02 0.04 1.00 -0.04 -0.09
Distance_km -0.02 -0.00 -0.01 -0.04 1.00 -0.10
Price 0.97 0.23 0.09 -0.09 -0.10 1.00
Area_sqft
Bedrooms
Bathrooms
HouseAge
Distance_km
Price
1.0
0.8
0.6
0.4
0.2
0.0

```
Area_sqft  Bedrooms  Bathrooms  HouseAge  Distance_km  Price

[6/7] HOUSE PRICE PREDICTION
======================================================================
Enter house details to estimate its price.
Area: 100 to 10000 square feet
Bedrooms: 1 to 20
Bathrooms: 1 to 15
House age: 0 to 150 years
Distance: 0 to 200 km

House area (sq ft): 444
Number of bedrooms: 5
Number of bathrooms: 6
House age (years): 89
Distance from city centre (km): 7

======================================================================
             HOUSE PRICE PREDICTION RESULT
======================================================================
Area: 444 sq ft
Bedrooms: 5
Bathrooms: 6
House age: 89 years
Distance: 7.00 km

Selected Model: Linear Regression
Estimated House Price: 100,455.12

Note: This is an estimate from synthetic training data,
not an actual market valuation.

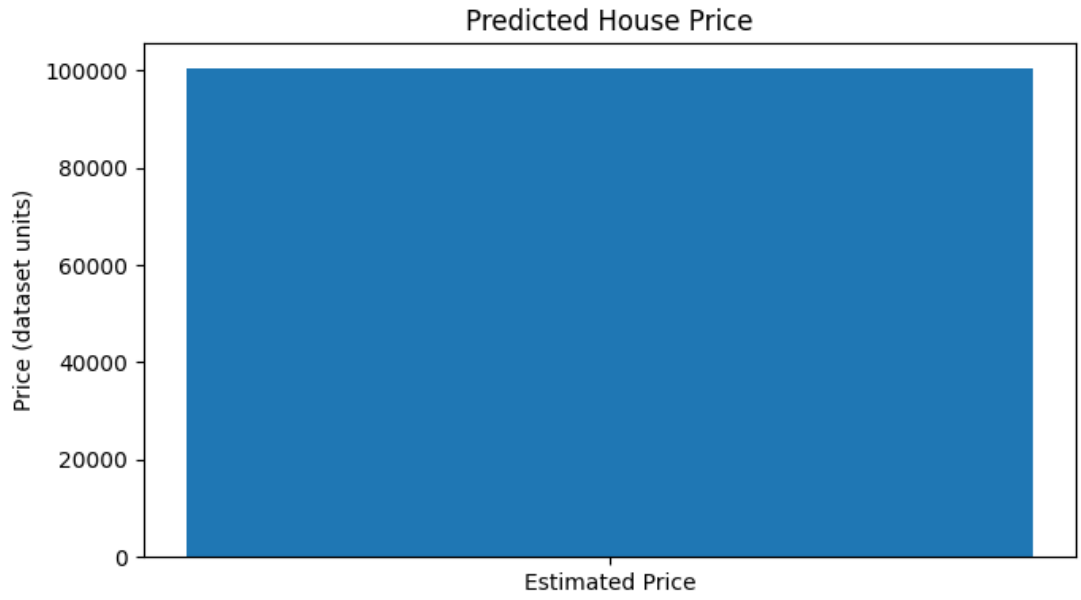
Prediction saved to house_price_prediction_result.csv
```

```
======================================================================
            HOUSE PRICE PREDICTION RESULT
======================================================================
Area: 444 sq ft
Bedrooms: 5
Bathrooms: 6
House age: 89 years
Distance: 7.00 km

Selected Model: Linear Regression
Estimated House Price: 100,455.12

Note: This is an estimate from synthetic training data,
not an actual market valuation.

Prediction saved to house_price_prediction_result.csv
```




```
[7/7] FINAL PROJECT SUMMARY
======================================================================
Dataset: house_prices_week6.csv
Models: Linear Regression, Decision Tree, Random Forest
Preprocessing: Duplicate removal and median imputation
Evaluation: MAE, MSE, RMSE, R2 Score
Visualization: Model comparison, prediction scatter plot,
               feature analysis and correlation heatmap
User interaction: House feature input and price prediction
```

8. Generated Files and Execution Notes

AI project files

- student_performance_week6.csv — synthetic student dataset.
- student_prediction_result.csv — generated after valid student input.

ML project files

- house_prices_week6.csv — synthetic house-price dataset.
- house_price_prediction_result.csv — generated after valid house input.
- model_evaluation_results.csv — regression model evaluation metrics.

Execution notes

- Install pandas, NumPy, Matplotlib, Seaborn, and scikit-learn in Colab before running the scripts.
- The scripts generate their own datasets; no CSV upload is required.
- Enter valid numeric values when the interactive input prompts appear.
- Download generated CSV files through the Colab Files panel if required.
- Paste real screenshots into the reserved output sections after running the code.
- Copy numerical metrics exactly from Colab; this report does not invent accuracy or prediction results.

Conclusion

The Week 6 report presents two complete demonstration workflows: an AI classification and recommendation system, and an ML regression prediction system. Both cover data preparation, model training, evaluation, visualization, and interactive predictions. Actual output screenshots and metrics should be added after successful execution in Google Colab.

Submission Details

Intern	**Saud Masood**
Program	DevSphere Internship Program
Domain	Artificial Intelligence & Machine Learning
Submission	Week 6 Final Project